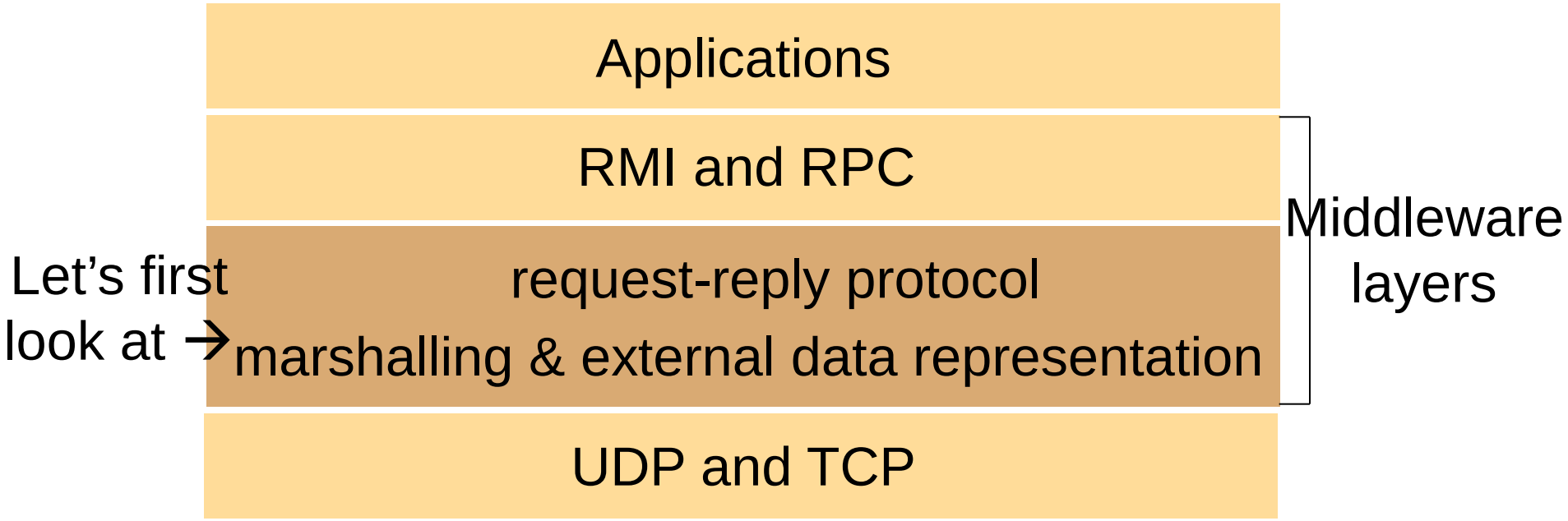
Fundamentals – Foundation

Interprocess Communication

Middleware Layers



Outline

- External Data Representation & Marshalling
- Client-Server Communication

Marshalling and Unmarshalling

- Irrespective of the communication form used, data are transmitted as a sequence of bytes
- Data structures (e.g., array, record, list, tree) must be flattened (converted to a sequence of bytes) before transmission and rebuilt on arrival
- Client and server may have different representations of the same data type
  - Big-endian ordering of integer: most significant byte comes first
  - Little-endian ordering of integer: least significant byte comes first
  - 1934 = 078E (big endian ordering) → 078E = 36359 (little endian ordering)

Marshalling and Unmarshalling

- External data representation
  - An agreed standard for representation of data structures and primitive values
- For interprocess communication
  - Define a standard form suitable for transmission
  - Marshalling: convert data items into the form suitable for transmission (at the source, structured and primitive data items → external data representation)
  - Unmarshalling: disassemble a message on arrival and restore data items (at the destination, external data representation → structured and primitive data items)
- Examples
  - CORBA's Common Data Representation (CDR)
  - Java's object serialization

CORBA's Common Data Representation

- CORBA: Common Object Request Broker Architecture (it is a middleware)
- CORBA's Common Data Representation (CDR) can represent all primitive and constructed data types that can be used as arguments and results of remote method invocations in CORBA

CORBA's Common Data Representation

- 15 primitive types: short (16-bit), long (32-bit), unsigned short, unsigned long, float (32-bit), double (64-bit), char, boolean, octet, ...
  - Big-/little-endian: transmit in the sender's ordering and the recipient translates if necessary
  - Floating-point: IEEE standard (sign + exponent + fractional part)
  - Characters: an agreed code set
  - Each primitive value is placed at an index in the sequence of bytes according to its size
    - A primitive value of size *n* bytes is appended to the sequence at an index that is a multiple of *n*

CORBA's Common Data Representation

- Constructed types:

Type	Representation
sequence	length (unsigned long) followed by elements in order
string	length (unsigned long) followed by characters in order
array	array elements in order (no length specified because it is fixed)
struct	in order of declaration of the components
enumerated	unsigned long (the values are specified by the order declared)
union	type tag followed by the selected member

Example CORBA CDR Message

CDR (flattened) form of a Person struct with value: {“Smith”, “London”, 1934}

```
struct Person{
    string name;
    string place;
    unsigned long year;};
```

index in sequence of bytes	← 4 bytes →
0 – 3	5
4 – 7	“Smit”
8 – 11	“h___”
12 – 15	6
16 – 19	“Lond”
20 – 23	“on__”
24 – 27	1934

length of string

padded to flush on word boundary

length of string (started at index 12 instead of 9)

unsigned long (started at index 24 instead of 22)

CORBA's Common Data Representation

- Types of data items are not given in CDR form
- It is assumed that the sender and recipient have common knowledge of the order and types of the data items in a message
- For example, for RMI or RPC, each method invocation passes arguments of particular types in predefined order, and the result is a value of a particular type
- Support a variety of programming languages

Java Object Serialization

- For use by Java only
- In Java, both objects and primitive data values may be passed as arguments and results of method invocations
- Serialization (synonym of marshalling in Java): the activity of flattening an object or a connected set of objects into a serialized form suitable for storing on disk or transmitting in a message
- Deserialization (synonym of unmarshalling in Java): restoring an object or a set of objects from their serialized form

Java Object Serialization

- Assumption: the process doing deserialization has no prior knowledge of the object types in the serialized form, so some information about the class of each object needs to be included in the serialized form

Example Java Serialized Form

```
public class Person
implements Serializable{
    private String name;
    private String place;
    private int year;};
```

Java serialized form of a Person instance with value: {“Smith”, “London”, 1934}

Person	8 byte version number		h0
3 (# of fields)	int year	java.lang. String name	java.lang. String place
1934	5 Smith	6 London	h1

class name and version number

number, type and name of instance variables

values of instance variables

h0 is a class handle and h1 is an object handle (usage to be discussed in the next example)

Java Object Serialization

- Format: class information, types and names of instance variables, values of instance variables
- If the instance variables belong to new classes, their class information must also be written out, followed by the types and names of their instance variables
- This recursive procedure continues until all necessary classes have been written out
- Each class is given a handle and no class is written more than once – the handles are written instead where necessary
  - Handles are references within the serialized form

Example Java Serialized Form

Java serialized form of a Couple instance with value: {{"Smith", "London", 1934}, {"Jones", "Paris", 1945}}

```
class Couple implements Serializable{
   private Person one;
   private Person two;
   public Couple(Person a, Person b) {
      one = a;  two = b;}};
```

Couple	8 byte version number		h0
2	Person one	Person two	
Person	8 byte version number		h1
3	int year	java.lang.String name	java.lang.String place
1934	5 Smith	6 London	h2
h1	1945	5 Jones	5 Paris
h3		h4	

class name, version number

number, type and name of instance variables

serialize instance variable "one" of Couple

values of instance variables

serialize instance variable "two" of Couple

values of instance variables

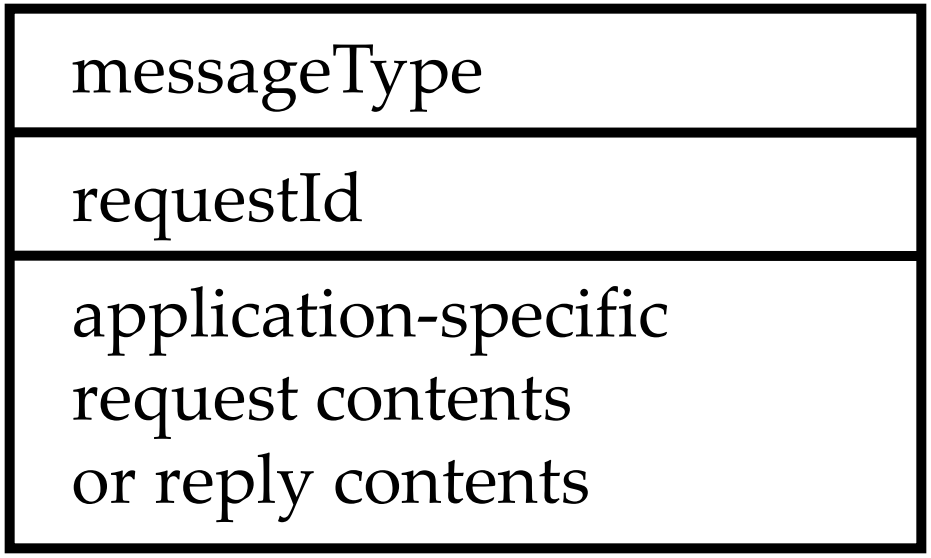
Summary of External Data Representations

- Need standards to facilitate communication and development of applications
- Standards cover a wide range: primitive data and structured data
- CORBA and its predecessors choose to marshal data for use by recipients that have prior knowledge of the types of its components
- Java object serialization includes full information about the types of its contents, allowing the recipient to reconstruct it purely from the contents

Outline

- External Data Representation & Marshalling
- Client-Server Communication

Request-Reply Message Structure

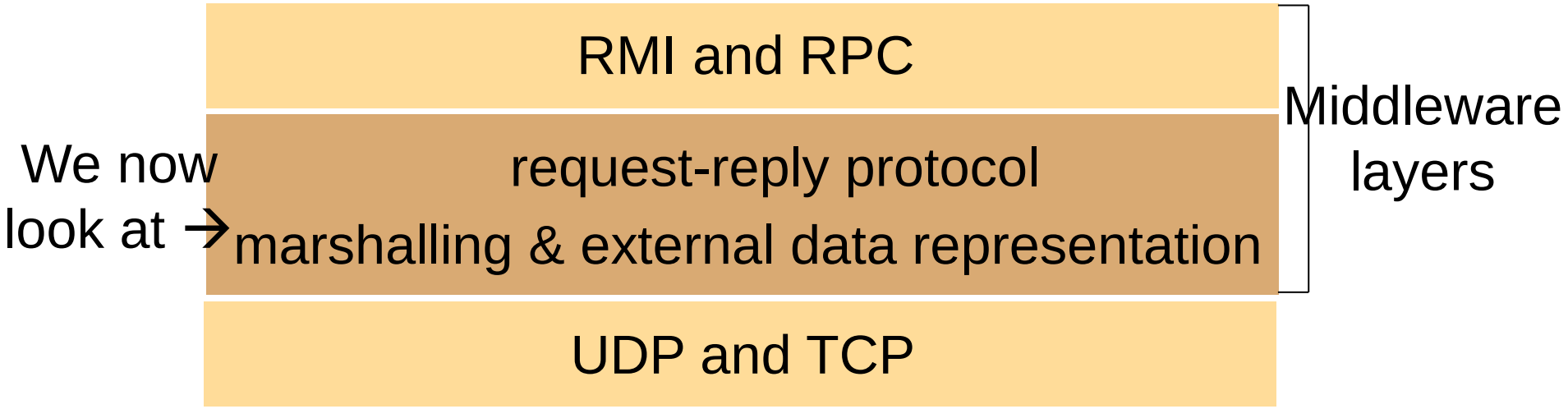


- Unique request message identifier:

  requestId (incremented each time a request is sent by the client process)

  + an identifier for the client process (e.g., Internet address + port number)

Interprocess Communication



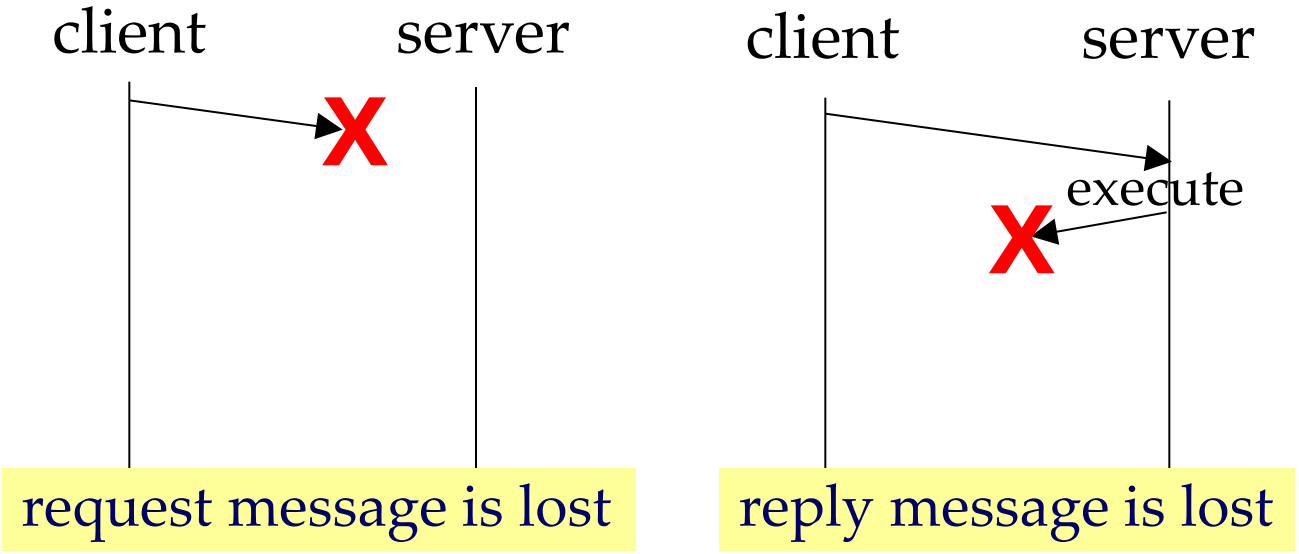
- We perceive a network supporting TCP and UDP
- Reliability
  - Validity: the message reaches the destination (the message in the outgoing message buffer of the sender is eventually delivered to the incoming message buffer of the receiver)
  - Integrity: the message received is identical to the one sent, and no message is delivered more than once

Request-Reply Protocol over UDP

- A message sent by UDP is transmitted without acknowledgements or retries
  - If a failure occurs, the message may not arrive
  - Messages larger than maximum allowable size must be fragmented for transmission
- UDP is not a reliable communication service
  - Integrity: use checksums to detect and reject corrupt packets
  - Validity: messages may be dropped occasionally (communication omission failures)
- Applications: any application that requires quick response (e.g., DNS requests), can assume reliability (e.g., on LAN), or handles faults itself

Request-Reply Protocol over UDP

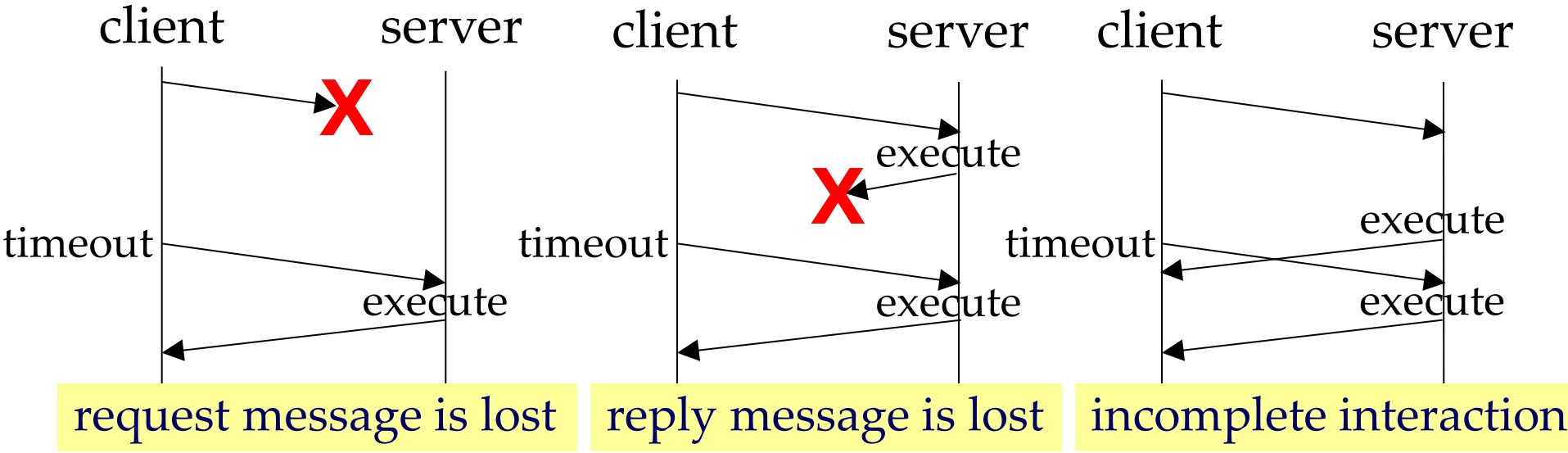
- UDP suffers from communication omission failures



- Goal: to build reliable request-reply protocols over UDP
- Other possible failures: processes may have crash and Byzantine failures (these are common to request-reply interactions over both UDP and TCP, will be discussed in later chapters)

Building Reliable Request-Reply Protocols over UDP

- Client uses a timeout when waiting for server's reply and sends the request repeatedly after a timeout



- Solve the problem of lost request message
- For lost reply message, operation is executed more than once
- Introduce a new problem of incomplete interaction, where the operation is also executed more than once

Building Reliable Request-Reply Protocols over UDP

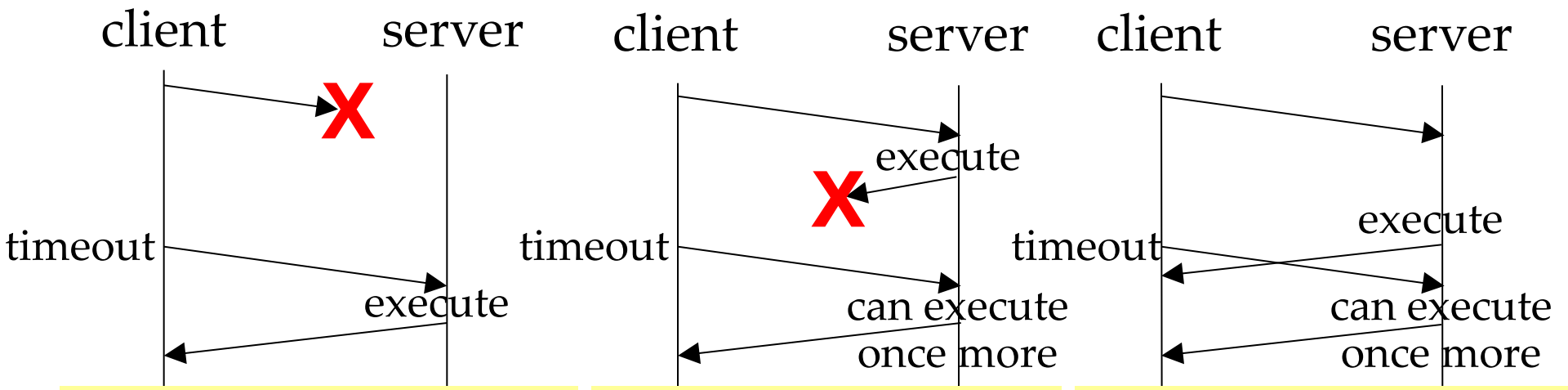
- Is repeated execution harmful?
  - Idempotent operations: those can be performed repeatedly with the same effect as if performed exactly once
    - e.g., adding an element to a set

      $\{1,2\} \cup \{3\} = \{1,2,3\}; \{1,2,3\} \cup \{3\} = \{1,2,3\}; \ldots\ldots$
    - e.g., checking the balance of a bank account
  - Non-idempotent operations: those if performed repeatedly have different effects from if performed exactly once
    - e.g., increasing a variable by 3

      $2 + 3 = 5;\ \ 5 + 3 = 8;\ \ 8 + 3 = 11; \ldots\ldots$
    - e.g., depositing money into a bank account

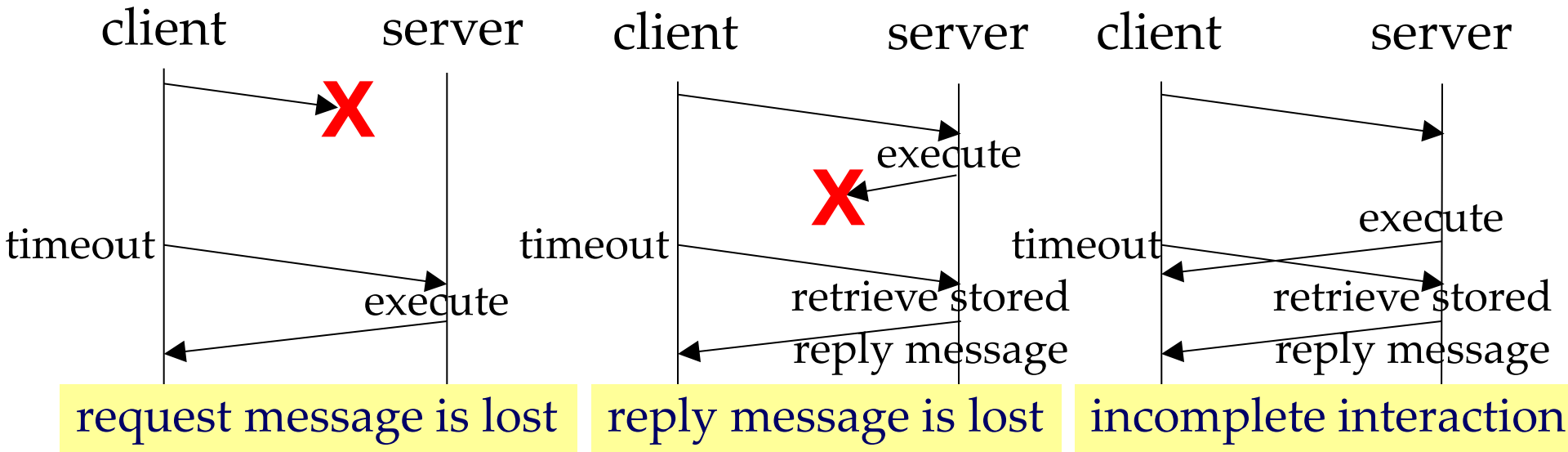
Building Reliable Request-Reply Protocols over UDP

- For idempotent operations
  - No need to avoid server executing the operations more than once for the same request
  - Execute operations and reply as usual for all requests



Building Reliable Request-Reply Protocols over UDP

- For non-idempotent operations
  - Server identifies messages from the same client with same requestId and filter out duplicates to avoid server executing an operation more than once for the same request
  - If an incoming request is seen for the first time, execute operation, reply as usual, and store the reply message (results) in a history
  - If an incoming request is a duplicate request, retrieve stored reply message from history and retransmit it to the client



Request-Reply Protocol over TCP

- TCP offers an abstraction of "reliable stream"
  - Connection-oriented: a connection must be setup before any data are transferred
  - Acknowledgements and retries
  - Transmit without worrying about message size, large messages get segmented in transmission transparently

Request-Reply Protocol over TCP

- TCP is a reliable communication service
  - Integrity: use checksums to detect and reject corrupt packets; use sequence numbers to detect and reject duplicate packets
  - Validity: successfully received packets are acknowledged; use timeouts and retransmissions to deal with lost packets
  - TCP ensures request and reply messages are delivered reliably

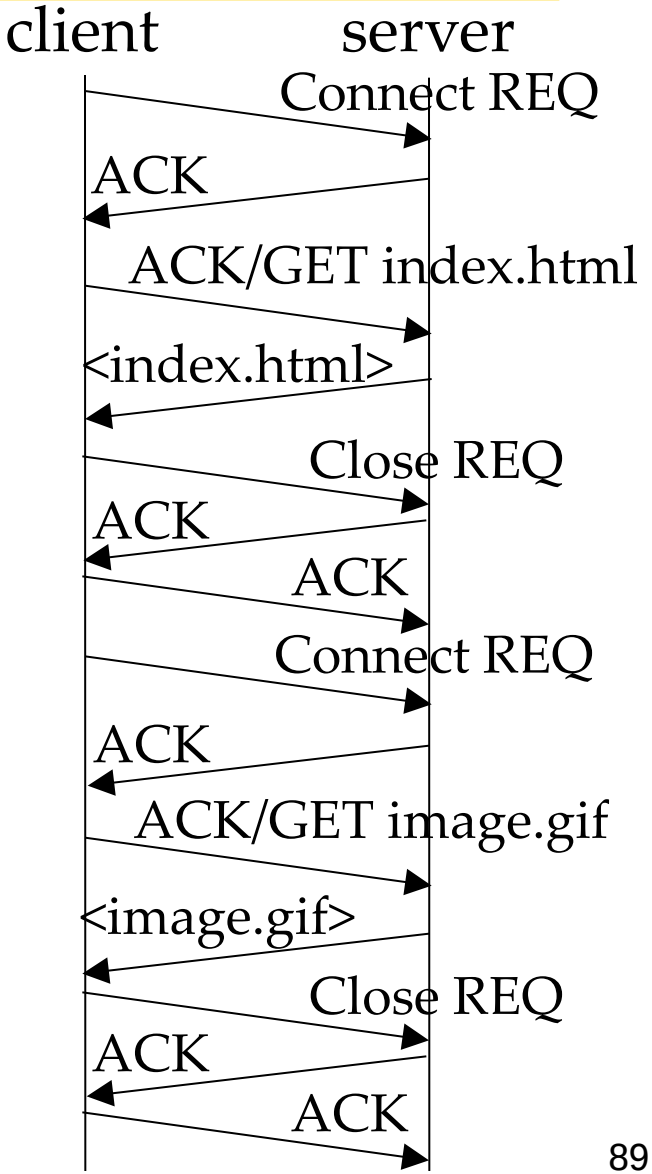
    → no need to deal with retransmission of messages, filtering of duplicates and maintaining histories in request-reply protocols over TCP

Request-Reply Protocol over TCP

- Overhead
  - Store state information at source and destination (e.g., to detect duplicate packets)
  - Transmission of extra messages (e.g., connect request, acknowledgement)
  - Additional latency (e.g., due to connection setup)
- Applications: any application that does not wish to handle missed/duplicated data or message sizes
  - Telnet, FTP, SMTP, HTTP, ……
- Our goal: to reduce overhead of request-reply protocols over TCP

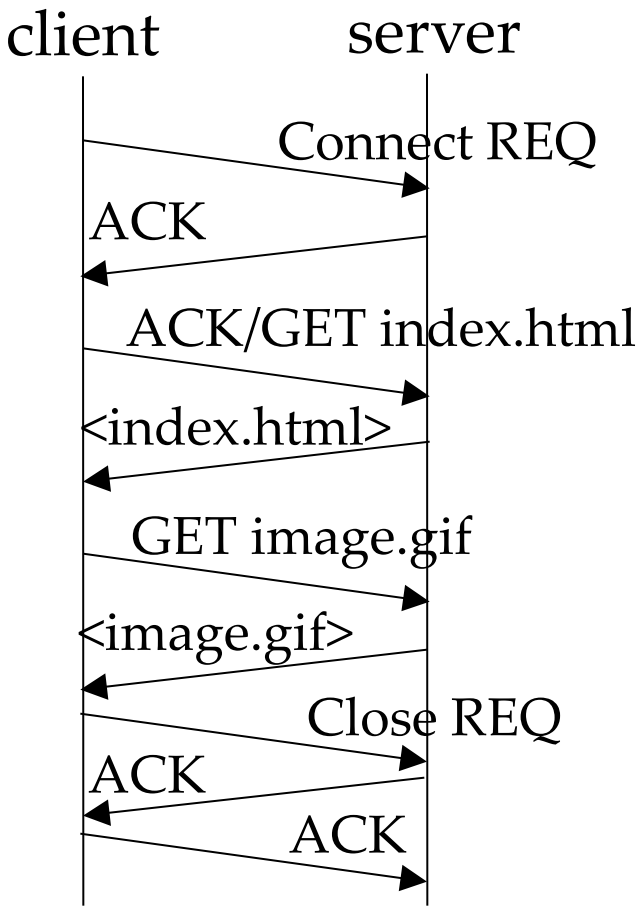
Reducing Overhead of Request-Reply Protocols over TCP

- Earlier version of HTTP sets up a new TCP connection for each HTTP request
  - Connection is closed on completion of the request
- Multiple pairs of requests and replies in a short duration can be sent over the same connection
  → see next slide



Reducing Overhead of Request-Reply Protocols over TCP

- Recent version of HTTP uses persistent connections that allow a client to reuse a single TCP connection to send multiple HTTP requests
  - Connection is kept alive on completion of a request
  - Advantage: amortizing the overhead of connection establishment over a group of requests
  - Advantage: avoiding multiple TCP slow-starts



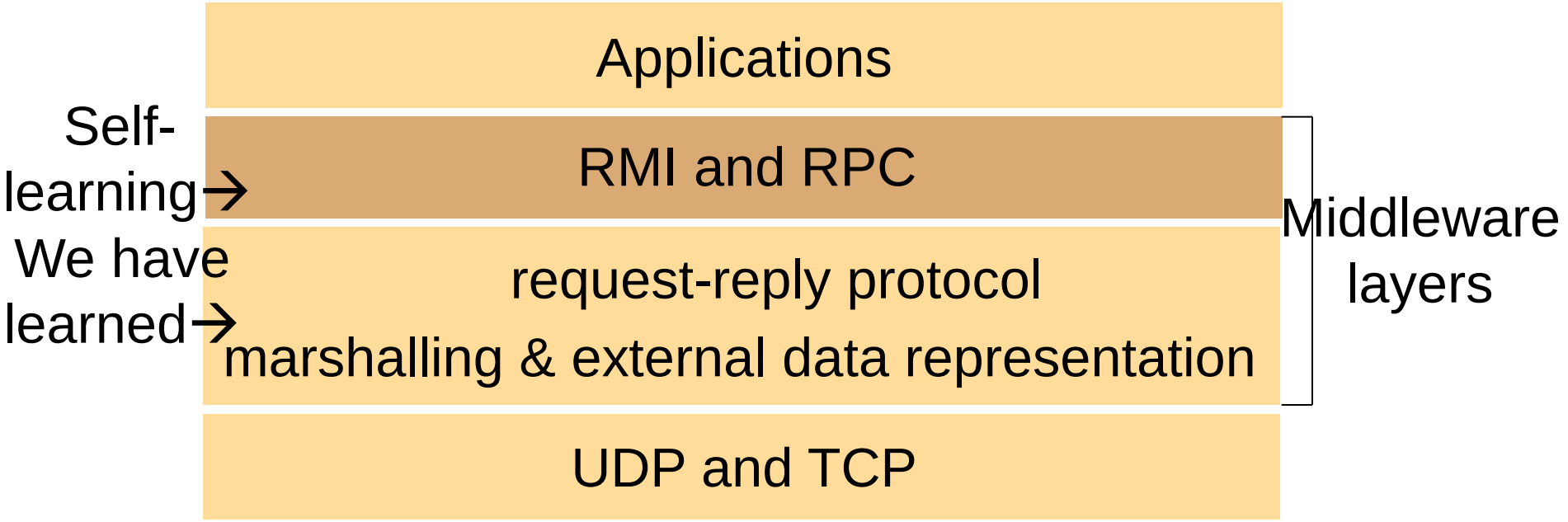
Summary

Learned about:

- External representations & marshalling (CDR and Java Serialization)
- Request-reply protocols for client-server communication

These are basis for building distributed systems and applications

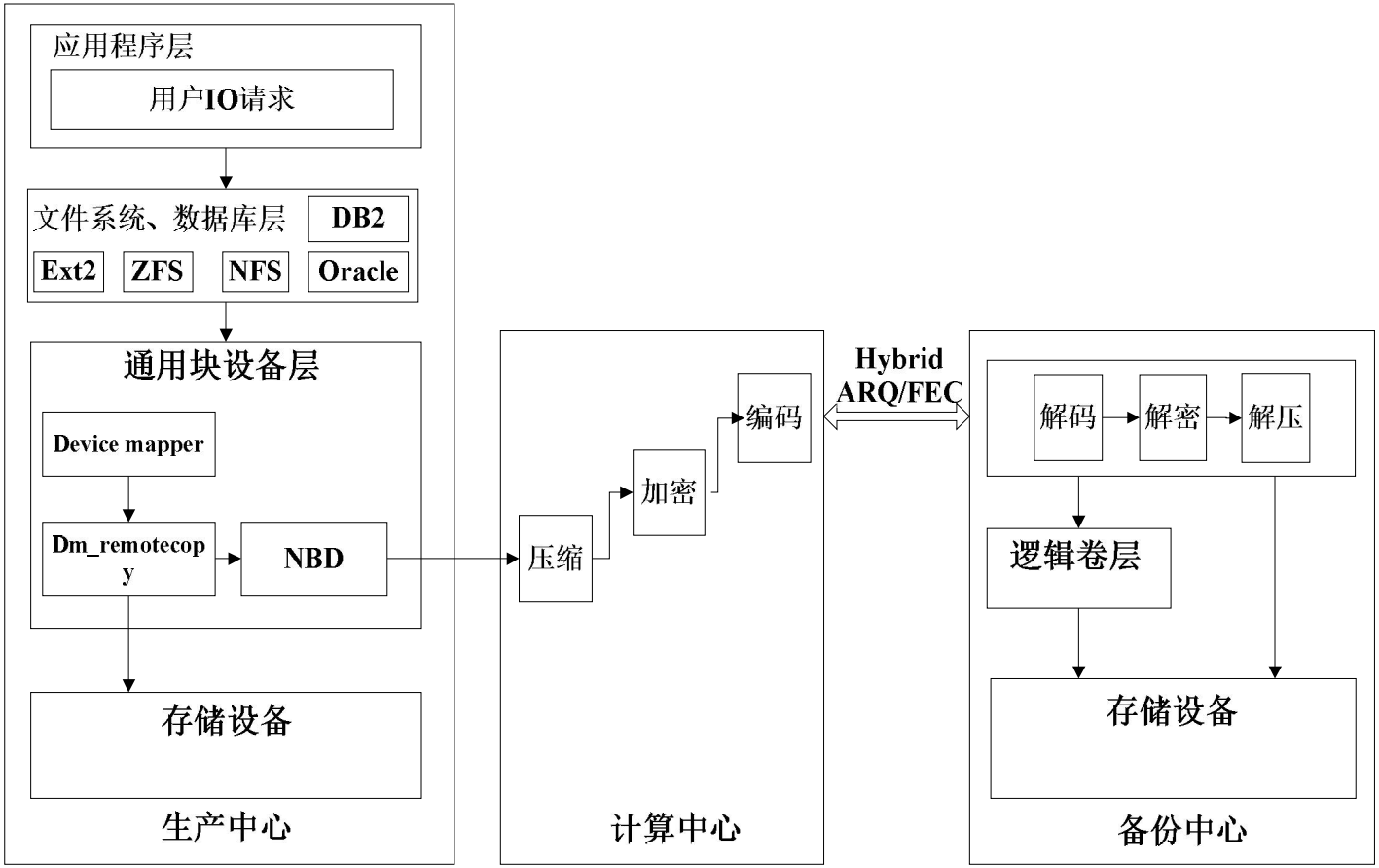
Middleware Layers



- Self-learning: distributed object model and remote method invocation
  - Reference book A, Sections 5.4 and 5.5

基于FEC的远程复制传输优化

- 实时远程复制



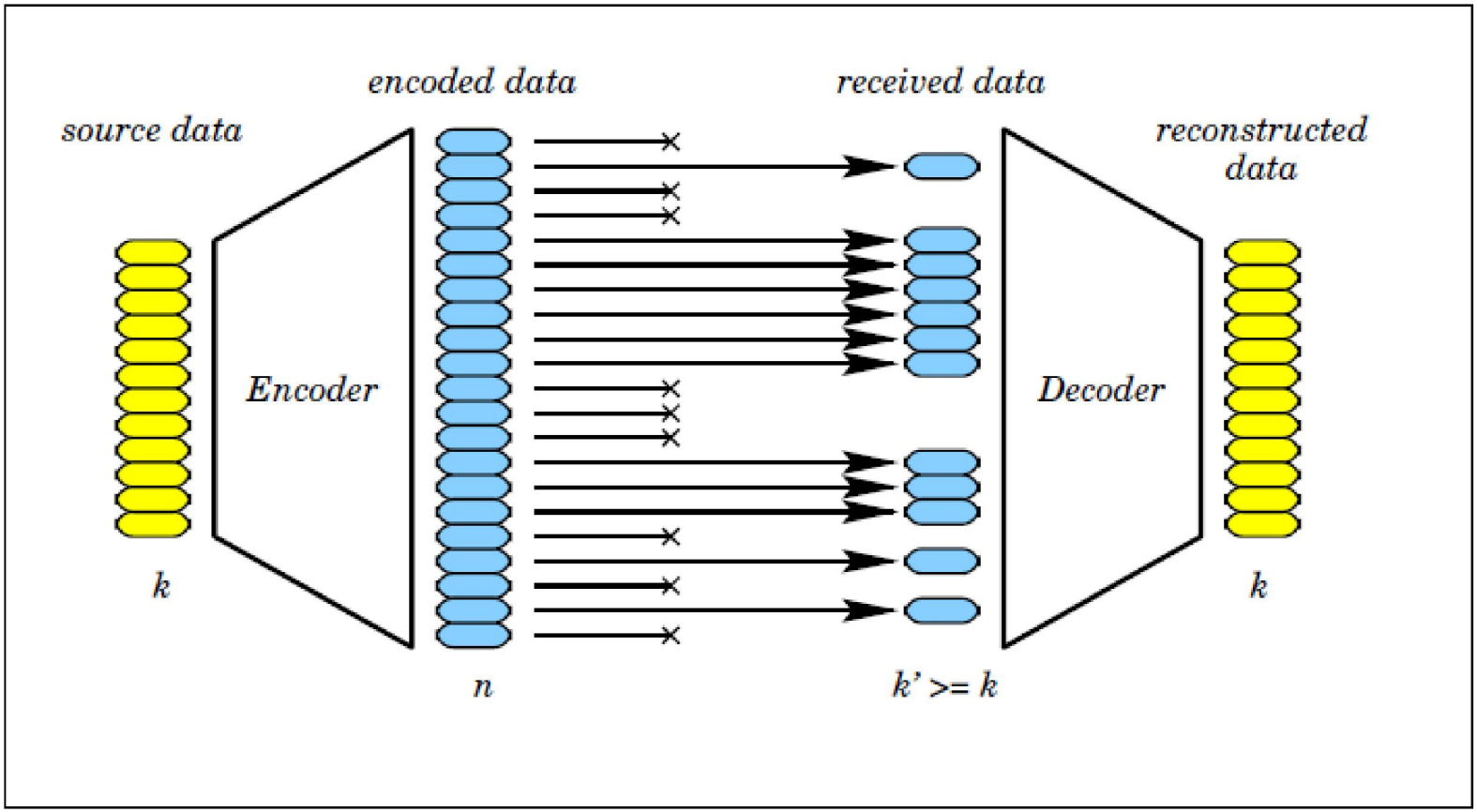
传输性能问题

- 广域网的一些特性
  - 设计传输协议时不可忽略RTT，范围一般在100到500ms之间
  - 存在着丢包现象，大部分丢包率均在0%到1%之间
  - 网络发送带宽有限
- TCP协议采用反馈重传机制（ARQ）
  - 被动地在出错时重传包

➔延迟大大增加，带宽利用率严重下降

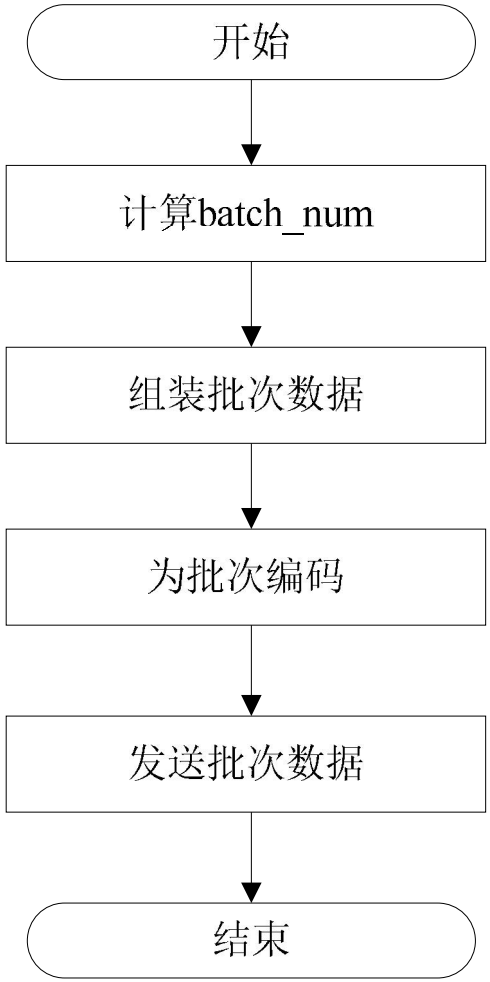
FEC机制

- **Forward error correction——变被动为主动**



Hybrid ARQ/FEC协议

- 计算中心处理流程

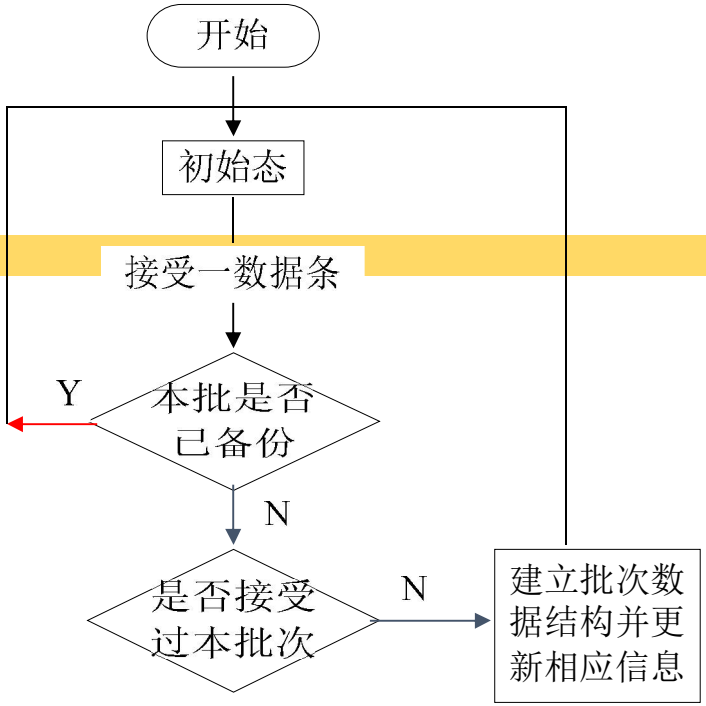


数据单元报文格式

batch-id	batch-num	seq	data

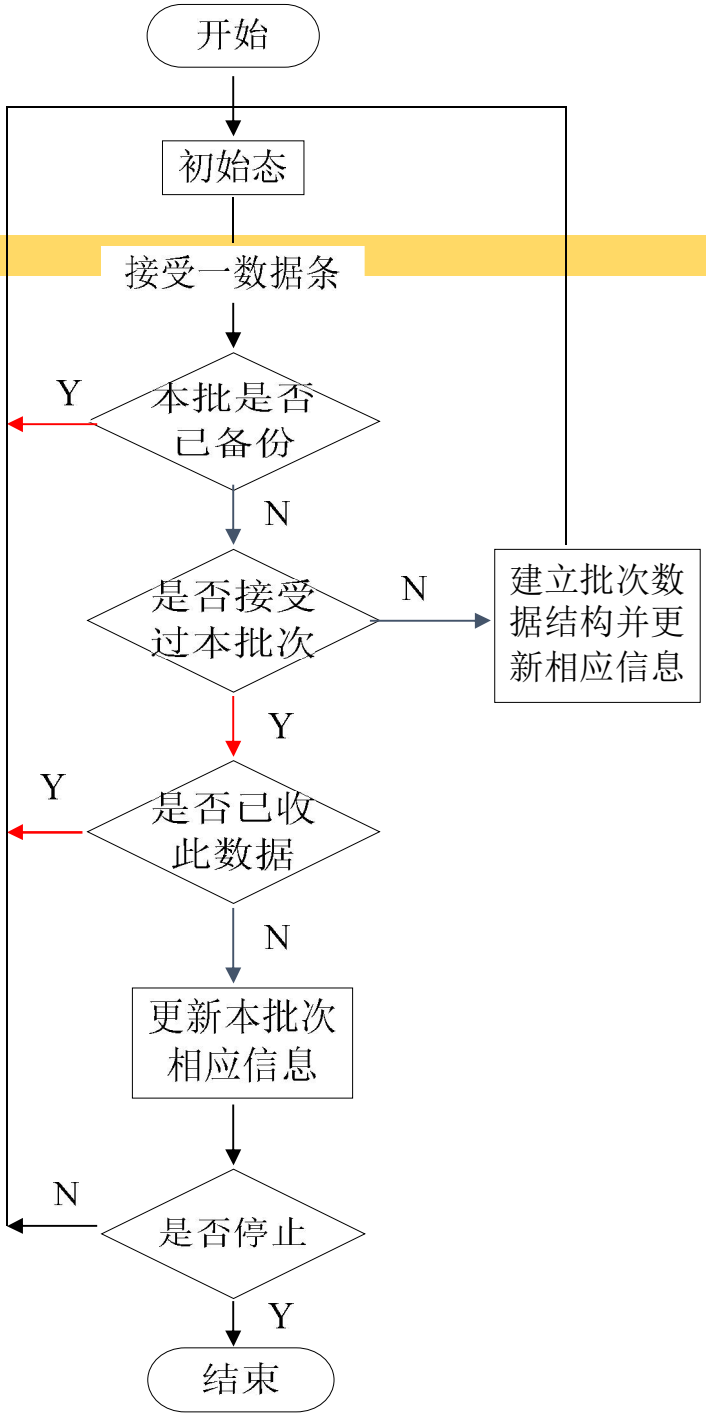
备份中心

- 接收数据流程



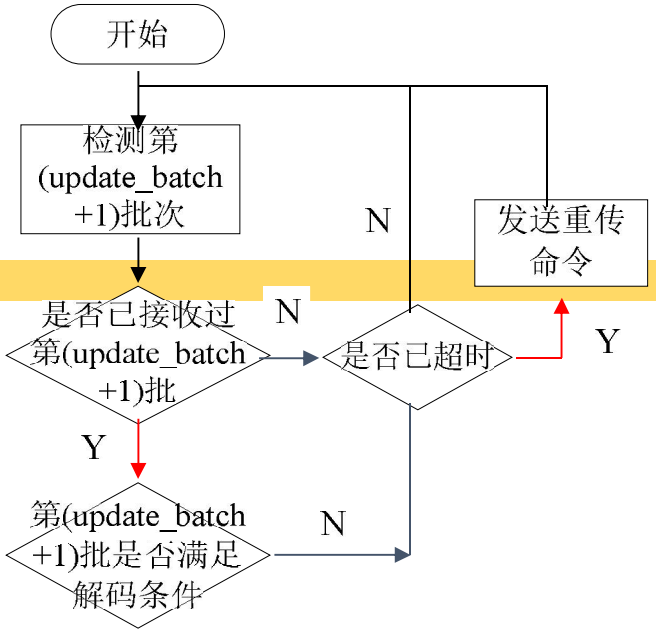
备份中心

- 接收数据流程


开始
初始态
接受一数据条
本批是否已备份
Y
N
是否接受过本批次
N
建立批次数据结构并更新相应信息
Y
是否已收此数据
Y
N
更新本批次相应信息
是否停止
N
Y
结束

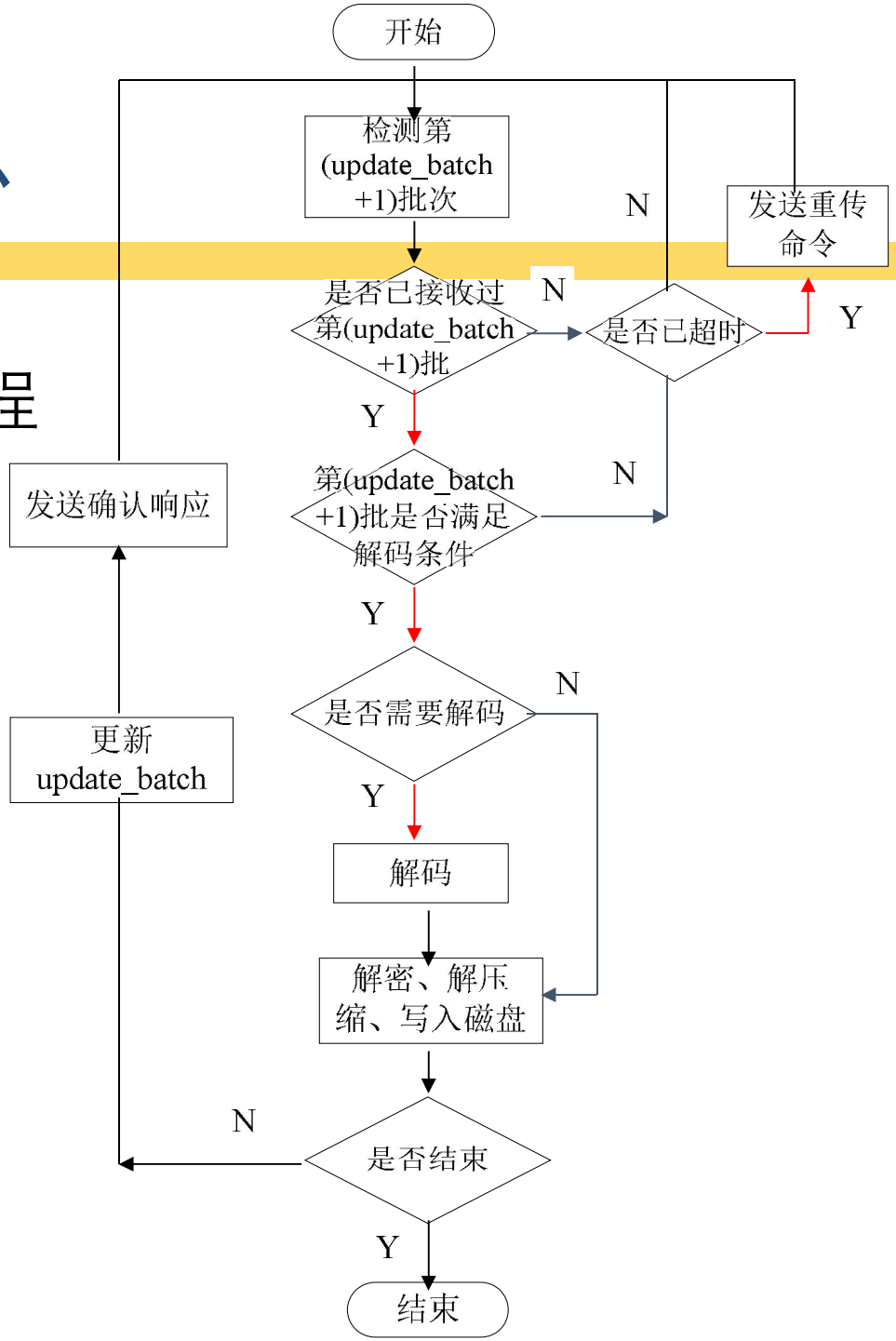
备份中心

- 备份数据流程


开始
检测第(update_batch+1)批次
是否已接收过第(update_batch+1)批
N
是否已超时
N
发送重传命令
Y
Y
第(update_batch+1)批是否满足解码条件
N

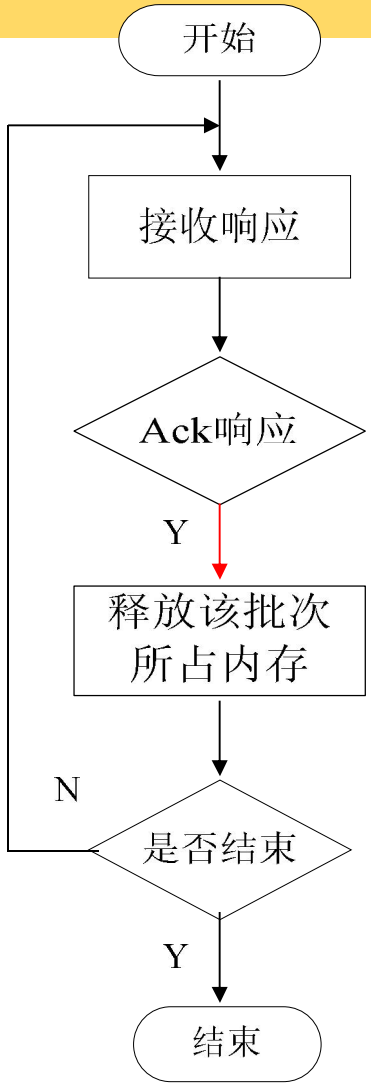
备份中心

- 备份数据流程


开始
检测第(update_batch+1)批次
是否已接收过第(update_batch+1)批
N
是否已超时
Y
N
发送重传命令
Y
第(update_batch+1)批是否满足解码条件
N
Y
是否需要解码
N
Y
解码
解密、解压缩、写入磁盘
是否结束
N
Y
结束
更新update_batch
发送确认响应

计算中心

- 响应流程

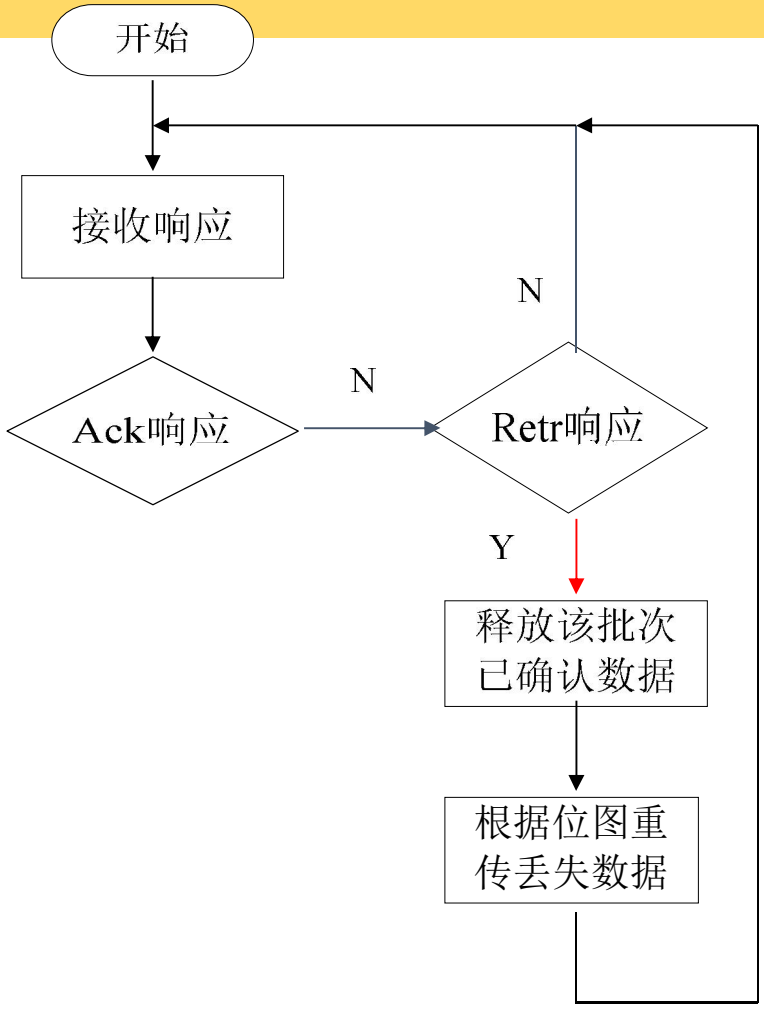


响应报文格式

batch-id	type	bitmap

计算中心

- 响应流程

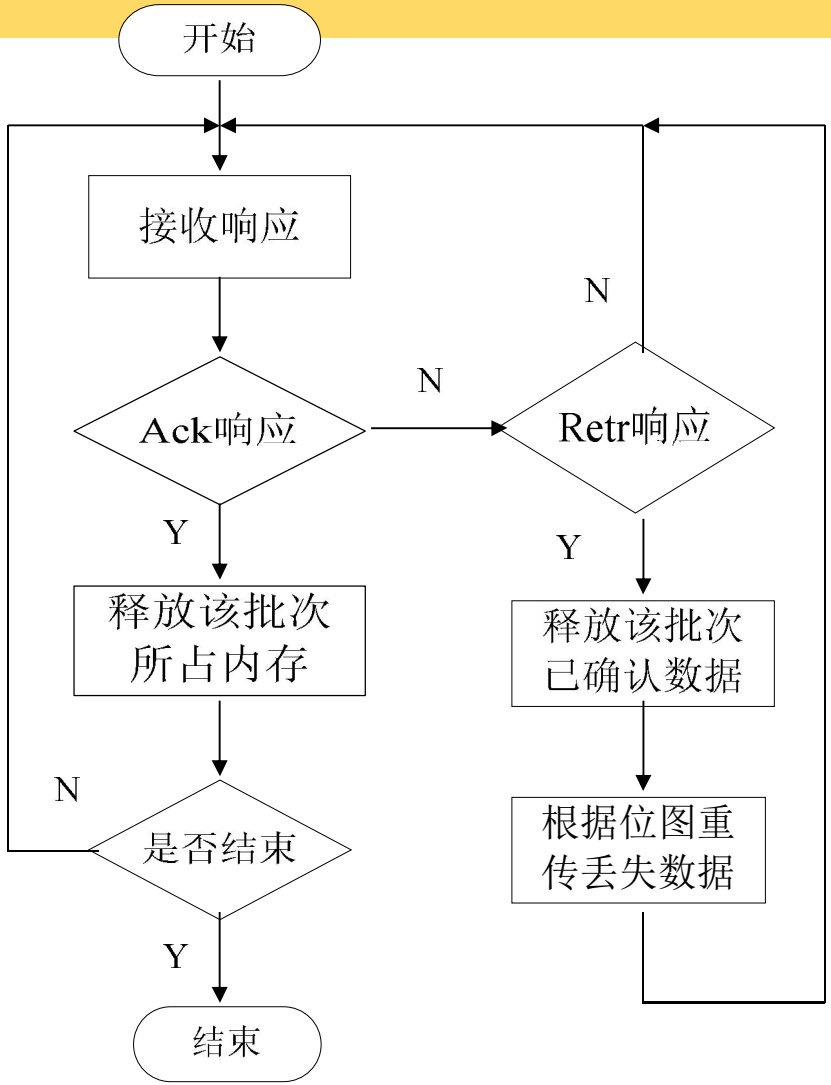


响应报文格式

batch-id	type	bitmap

计算中心

- 响应流程

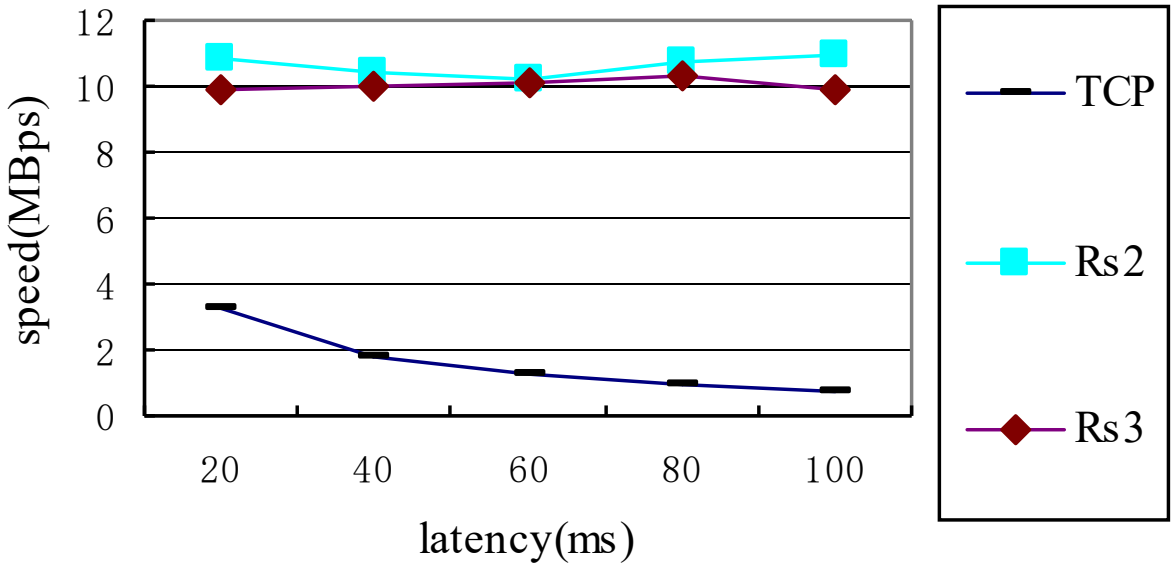


响应报文格式

batch-id	type	bitmap

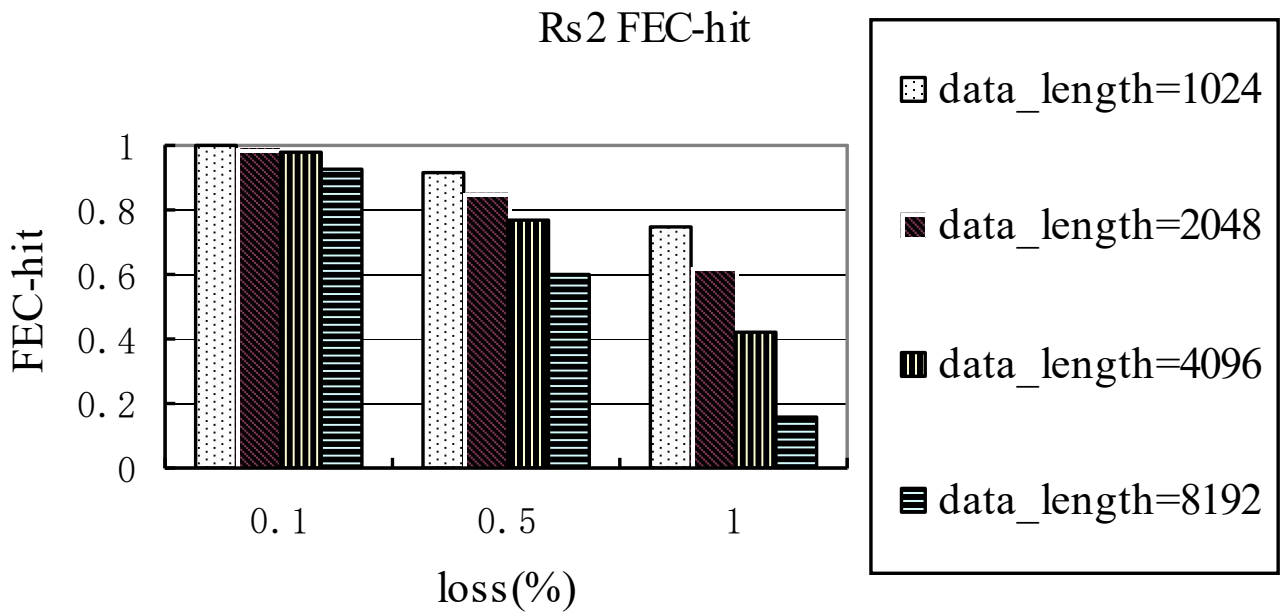
性能测试

- 延迟对备份性能的影响



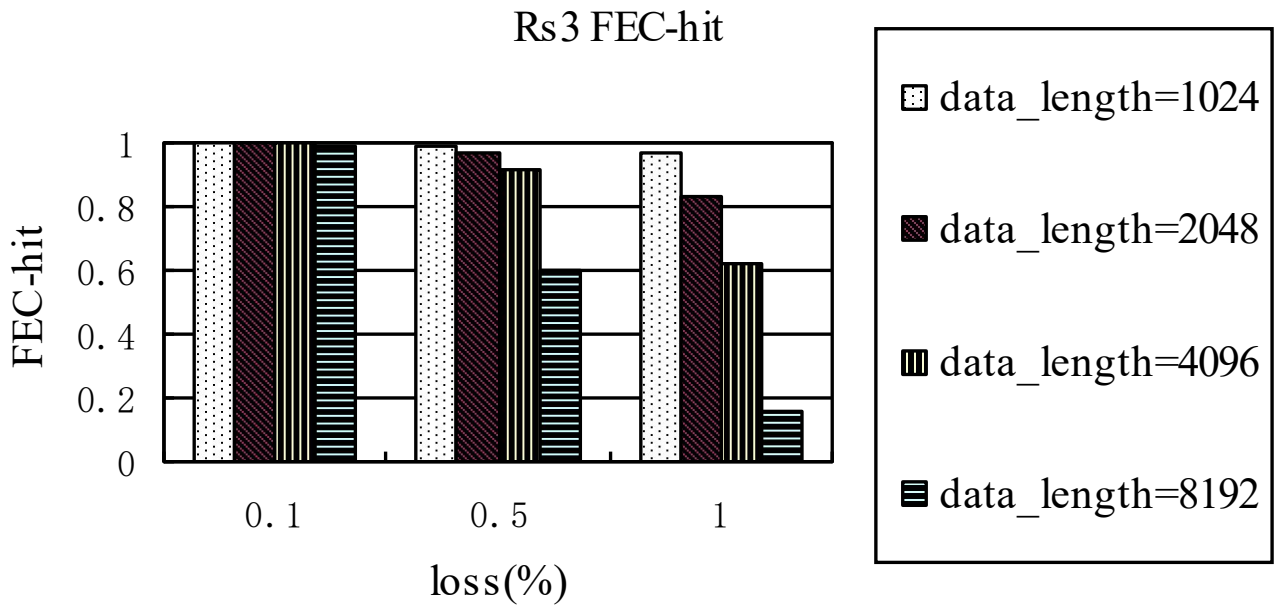
性能测试

- 丢包率和数据长度对备份性能的影响



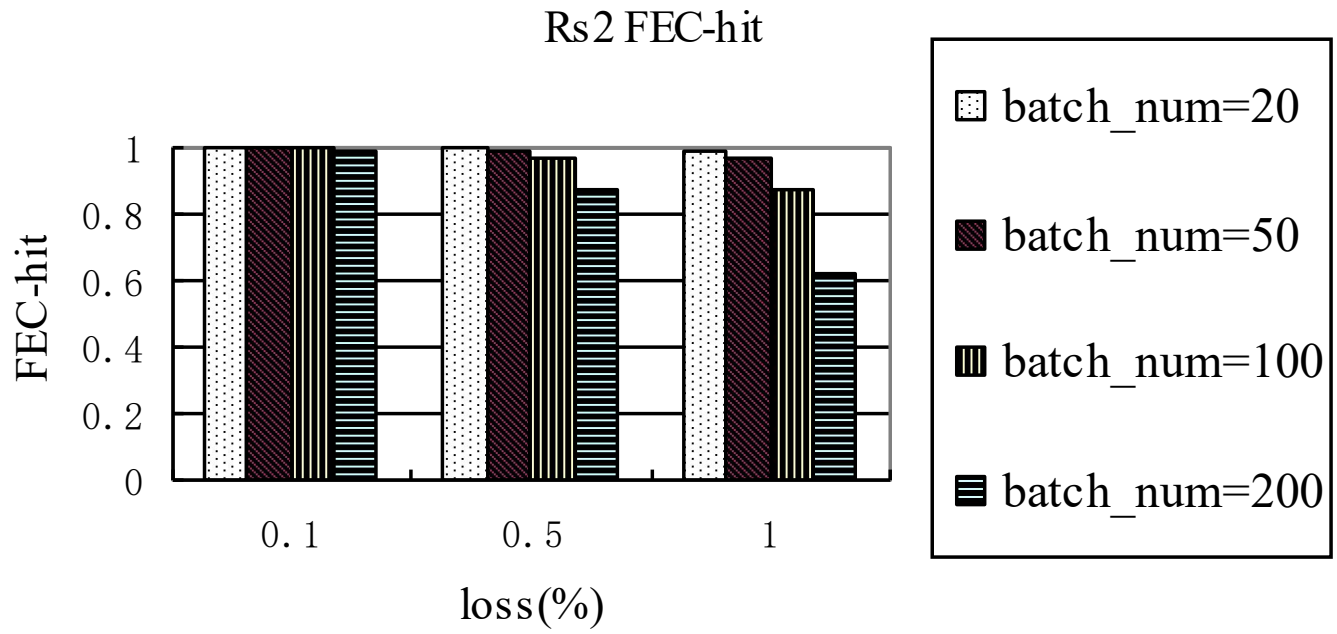
性能测试

- 丢包率和数据长度对备份性能的影响



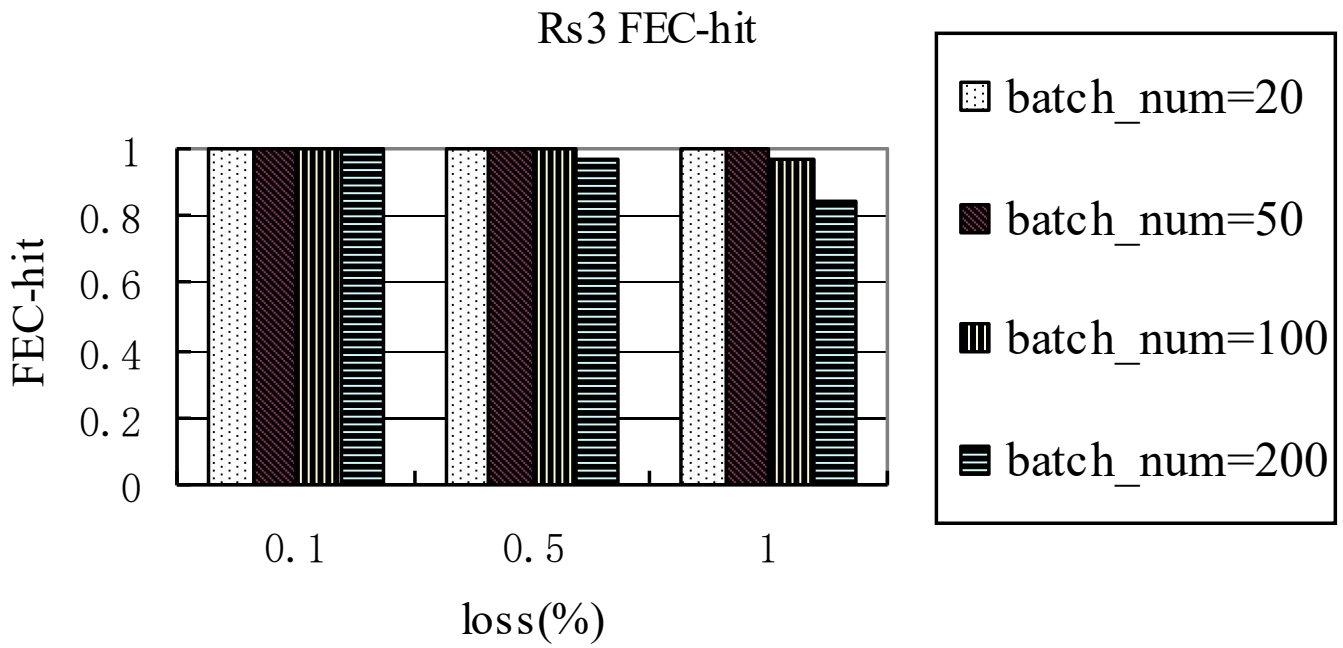
性能测试

- 丢包率和批次大小对备份性能的影响



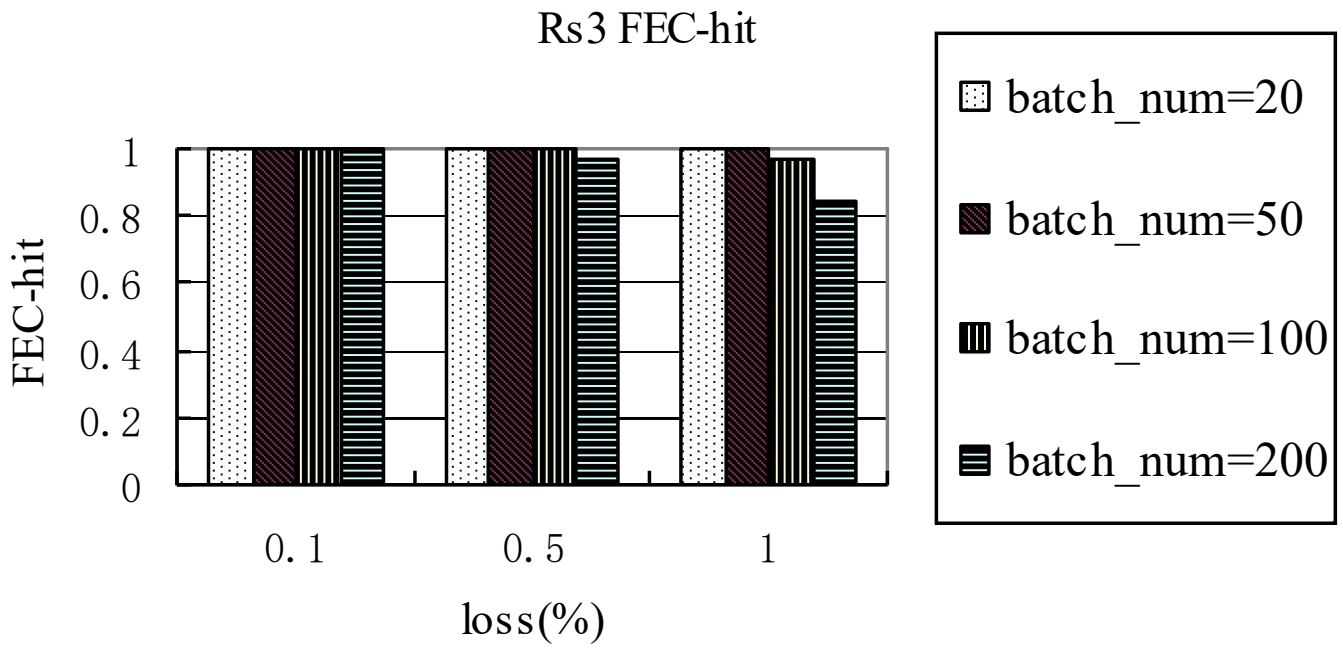
性能测试

- 丢包率和批次大小对备份性能的影响



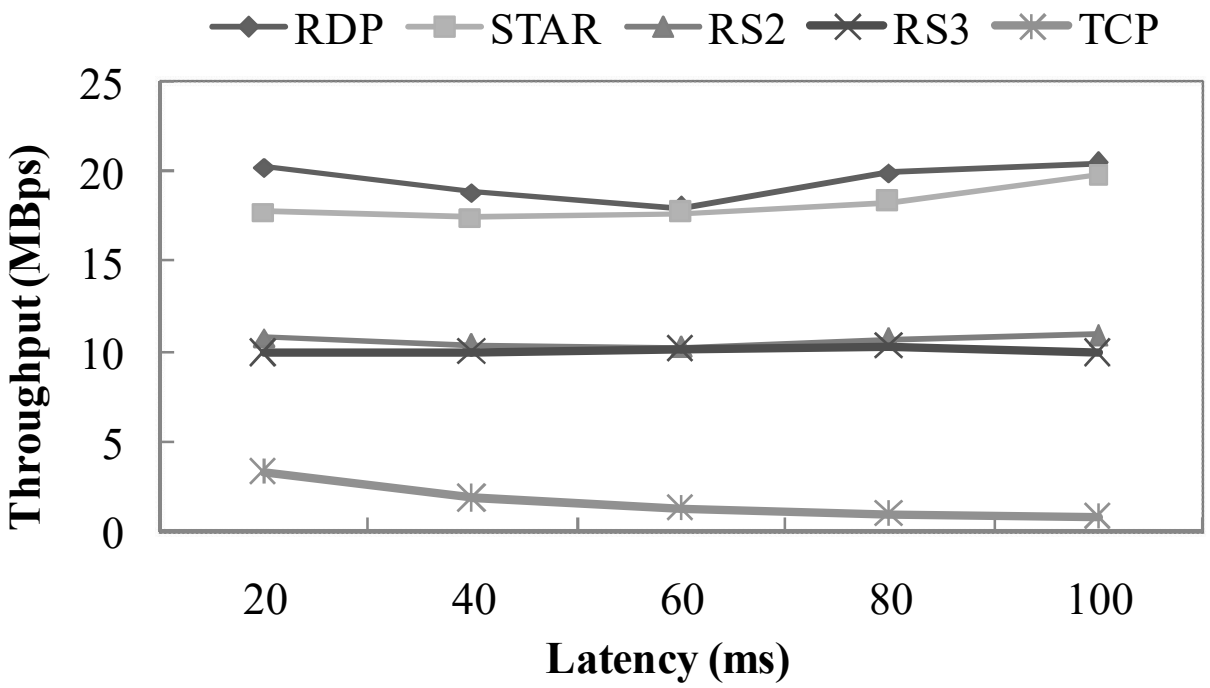
性能测试

- 丢包率和批次大小对备份性能的影响



性能测试

- 高效编码方案对备份性能的影响



性能测试

- 高效编码方案对备份性能的影响